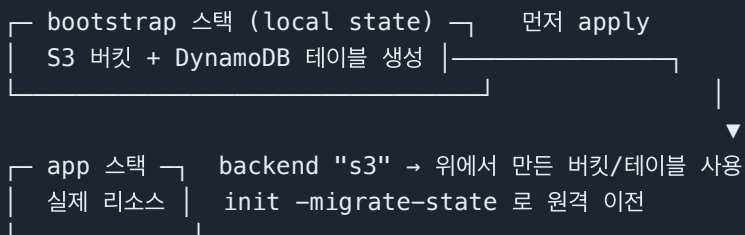


## Week 3. Remote Backend (S3 + DynamoDB, 락)

### 개념 오늘의 목표 & 핵심 순간 ★ (닭-달걀 문제)

지금까지 `tfstate` 는 로컬 파일이었다. 팀 협업 시 문제: state를 공유 못 하고, 동시에 `apply` 하면 충돌한다. → 해결: state를 **S3에 저장**하고 **DynamoDB로 락**을 건다. 그런데 **역설**: state를 담을 S3 버킷/DynamoDB 테이블 자체는 어디서 만들지? backend가 아직 없는데? → 이게 **닭-달걀(chicken-egg)** 문제. 그래서 **2-스택 패턴**으로 푼다.



### 실습 1. bootstrap 스택으로 backend 저장소 만들기

week3/bootstrap/main.tf :

```
terraform {
  required_version = ">= 1.9"
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
}
provider "aws" { region = "ap-northeast-2" }

resource "aws_s3_bucket" "state" {
  bucket = "boaz26-tfstate-kdh1834" # ← 유일한 이름
}
resource "aws_s3_bucket_versioning" "state" {
  bucket = aws_s3_bucket.state.id
  versioning_configuration { status = "Enabled" }
}
resource "aws_dynamodb_table" "lock" {
  name           = "boaz26-tf-lock"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key      = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

```
cd week3/bootstrap
terraform init && terraform apply
```

## 확인 bootstrap 완료

S3 버킷(버전관리 ON) + DynamoDB 테이블 `boaz26-tf-lock` 생성. 이 스택의 state는 로컬로 남겨둔다(이건 원격화 안 함).

## 실습 2. app 스택 — 로컬 state로 먼저 리소스 생성

week3/app/main.tf :

```
terraform {
  required_version = ">= 1.9"
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
}
provider "aws" { region = "ap-northeast-2" }

resource "aws_s3_bucket" "demo" {
  bucket = "boaz26-w3-app-kdh1834" # ← 유일한 이름
}
```

```
cd ../app
terraform init && terraform apply # 우선 로컬 tfstate로 생성
```

## 실습 3. backend 블록 추가 → 원격으로 마이그레이션 ★

week3/app/main.tf 의 `terraform { ... }` 안에 backend 추가:

```
terraform {
  required_version = ">= 1.9"
  backend "s3" {
    bucket      = "boaz26-tfstate-kdh1834" # bootstrap에서 만든 버킷
    key         = "week3/app/terraform.tfstate"
    region      = "ap-northeast-2"
    dynamodb_table = "boaz26-tf-lock"
    encrypt     = true
  }
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
}
```

```
terraform init -migrate-state # "yes" → 로컬 state를 S3로 복사
```

## 확인 마이그레이션 성공

- Successfully configured the backend "s3"! Terraform will automatically use this backend...
- S3 버킷 안에 `week3/app/terraform.tfstate` 객체 생성됨
- 로컬 `terraform.tfstate` 는 이제 비어있고 backup 파일만 남음

## 관찰 🕒 락 동작 확인

터미널 A에서 `terraform plan` 을 실행한 채(또는 `apply` yes 직전 대기), 터미널 B에서 `terraform plan` 실행  
→

- B에서 본 메시지: `Error acquiring the state lock` (\_\_\_에 기록)
- DynamoDB 테이블에 잠깐 생겼다 사라지는 `LockID` 항목: 봤다/못 봤다 \_\_\_

## 함정 AccessDenied on backend

IAM 사용자에게 S3 버킷 `Get/Put/List` + DynamoDB `GetItem/PutItem/DeleteItem` 권한 필요. 실습은 관리자 권한이면 통과.

참고: Terraform 최신 버전은 S3 backend의 `use_lockfile = true` 로 DynamoDB 없이도 락이 가능하지만, 오늘은 원리 이해를 위해 DynamoDB 방식을 쓴다.

## 정리 destroy — 순서 주의 ⭐

```
# 1) app 먼저 (원격 state 사용 중)
cd week3/app && terraform destroy
# 2) bootstrap 마지막 (backend 저장소를 지움)
cd ../bootstrap && terraform destroy
```

- ☐ app 리소스 삭제 → S3의 state 객체는 아직 남음
- ☐ bootstrap destroy로 상태 버킷·DynamoDB 삭제 (버킷 비어야 삭제됨)
- ☐ 순서를 반대로 하면 backend 저장소가 먼저 사라져 app을 못 지움 → 오늘의 교훈